

Rastreador de Hábitos

Manual del Proyecto

PWA con Spring Boot 4.1.0 + H2 + JPA

1. ¿Qué es este proyecto?

Rastreador de Hábitos es una aplicación web progresiva (PWA) que permite crear hábitos personales (por ejemplo "Leer", "Ejercicio", "Tomar agua") y marcar día a día si se cumplieron o no, con una visualización tipo cuadrícula similar al calendario de contribuciones de GitHub.

La aplicación es instalable en el celular o la computadora como si fuera una app nativa, funciona sin conexión a internet para la interfaz, y guarda todos los datos de forma persistente en una base de datos local.

1.1 Funciones principales

- Crear hábitos nuevos con nombre y color personalizado.
- Marcar/desmarcar el cumplimiento de un hábito en cualquier día de los últimos 30 días.
- Ver una cuadrícula visual de 30 días por hábito, coloreada según los días cumplidos.
- Cálculo automático de la racha actual (días consecutivos cumplidos hasta hoy).
- Eliminar hábitos (borra también todo su historial).
- Instalación como app (PWA) desde el navegador.
- Funcionamiento offline de la interfaz mediante Service Worker.

2. Arquitectura del proyecto

El proyecto combina un backend en Spring Boot con un frontend simple en HTML, CSS y JavaScript puro, sin frameworks de frontend. Spring Boot no solo expone la API, sino que también sirve los archivos estáticos de la interfaz.

2.1 Estructura de carpetas

```
habit-tracker/
├── src/main/java/com/isai/habittracker/
│   ├── HabitTrackerApplication.java
│   ├── model/
│   │   ├── Habit.java
│   │   └── HabitLog.java
│   ├── repository/
│   │   ├── HabitRepository.java
│   │   └── HabitLogRepository.java
│   └── controller/
│       └── HabitController.java
├── src/main/resources/
│   ├── application.properties
│   ├── static/
│   │   ├── index.html
│   │   ├── manifest.json
│   │   ├── sw.js
│   │   ├── offline.html
│   │   └── icons/
│   └── data/
│       └── habitdb.mv.db (se genera solo al correr el proyecto)
```

2.2 Capas del backend

- Model (Habit, HabitLog): representan las tablas de la base de datos. Un Habit tiene muchos HabitLog.
- Repository: interfaces que dan acceso a la base de datos sin necesidad de escribir SQL (Spring Data JPA).
- Controller: expone los endpoints HTTP que el frontend consume (API REST bajo /api).
- Frontend (static/): la interfaz visual, que se comunica con el backend usando fetch() en JavaScript.

2.3 Tecnologías usadas

- Spring Boot 4.1.0 — framework del backend.
- Spring Web — expone la API REST y sirve los archivos estáticos.
- Spring Data JPA — comunicación con la base de datos mediante objetos Java.
- H2 Database — base de datos embebida, guardada en un archivo local (./data/habitdb).
- Lombok — reduce código repetitivo (getters/setters) en las entidades.
- HTML/CSS/JavaScript puro — interfaz, sin librerías externas.
- Service Worker + Web App Manifest — lo que convierte la interfaz en una PWA instalable.

3. Cómo correr el proyecto

3.1 Requisitos

- Java 17 instalado.
- Apache NetBeans 15 (o cualquier IDE, o solo terminal).

3.2 Iniciar el servidor

Se recomienda iniciar el proyecto desde la terminal dentro de la carpeta del proyecto, ya que el botón Run de NetBeans puede presentar conflictos con el plugin exec-maven-plugin:

```
cd habit-tracker
mvnw.cmd spring-boot:run
```

Cuando la consola muestre un mensaje similar a "Started HabitTrackerApplication", el servidor está listo.

3.3 Abrir la aplicación

Con el servidor corriendo, abrir en el navegador:

```
http://localhost:8080
```

3.4 Instalar como aplicación (PWA)

- En Chrome o Edge, buscar el ícono de instalación en la barra de direcciones, o abrir el menú (:) y seleccionar "Instalar Rastreador de Hábitos".

- En celulares Android, el navegador puede mostrar un aviso de "Agregar a pantalla de inicio".

4. Cómo usar la aplicación

4.1 Crear un hábito

- Escribir el nombre del hábito en el campo de texto (ejemplo: "Ejercicio").
- Elegir un color con el selector de color.
- Presionar el botón "Agregar".
- El hábito aparecerá como una tarjeta nueva con una cuadrícula de 30 días en gris.

4.2 Marcar un día como cumplido

- Dentro de la tarjeta de un hábito, hacer clic sobre el cuadrado correspondiente al día deseado.
- El cuadrado cambiará al color asignado al hábito, indicando que ese día se cumplió.
- Hacer clic de nuevo sobre el mismo cuadrado lo desmarca (vuelve a gris).

4.3 Ver la racha

Junto al nombre de cada hábito se muestra el número de días consecutivos cumplidos, contando hacia atrás desde el día actual (por ejemplo: "🔥 5 días"). Si se rompe la racha (un día sin marcar), el contador vuelve a cero.

4.4 Eliminar un hábito

Presionar el botón ✕ en la esquina de la tarjeta del hábito. Se pedirá una confirmación antes de borrar, ya que esta acción elimina también todo el historial de días marcados de ese hábito.

5. Referencia de la API (endpoints)

Todos los endpoints están disponibles bajo el prefijo /api. Estos son los que utiliza el frontend, y también se pueden probar directamente con herramientas como Postman.

Método	Ruta	Descripción
GET	/api/habits	Devuelve la lista de todos los hábitos creados.
POST	/api/habits	Crea un hábito nuevo. Cuerpo: { name, color }
DELETE	/api/habits/{id}	Elimina un hábito y todo su historial.
GET	/api/habits/{id}/logs	Devuelve el historial de días marcados de un hábito.
POST	/api/habits/{id}/toggle	Marca o desmarca un día. Cuerpo: { date: "YYYY-MM-DD" }

6. Base de datos

Se utiliza H2 en modo archivo (no en memoria), por lo que los datos persisten aunque se cierre y reinicie el servidor. El archivo se genera automáticamente en:

```
./data/habitedb.mv.db
```

Las tablas se crean y actualizan automáticamente al iniciar el servidor gracias a la configuración de Hibernate, sin necesidad de escribir sentencias SQL manualmente.

6.1 Tablas

- habits — id, name, color.
- habit_logs — id, habit_id, date, completed. Cada combinación de habit_id y date es única, para evitar duplicados.

7. Funcionamiento como PWA

7.1 manifest.json

Define el nombre, colores, ícono y modo de visualización (pantalla completa, sin barra del navegador) que usa el sistema operativo al instalar la app.

7.2 Service Worker (sw.js)

- Al instalarse, guarda en caché los archivos base (index.html, manifest.json, offline.html) para que la interfaz cargue sin conexión.
- Las peticiones a /api siempre se intentan por red primero, ya que requieren datos actualizados; si no hay conexión, devuelven un mensaje de error controlado.
- El resto de los archivos se sirven primero desde caché, y si no están disponibles ni en caché ni en red, se muestra offline.html.

8. Solución de problemas comunes

El botón Run de NetBeans falla con "Could not find or load main class \${start-class}"

Ejecutar el proyecto desde terminal con mvnw.cmd spring-boot:run, o corregir la acción "Run project" en las propiedades del proyecto (Properties → Actions) cambiando el goal a spring-boot:run.

Al presionar "Agregar" no pasa nada

Generalmente se debe a un error de referencia circular al convertir los datos a JSON (Habit contiene una lista de HabitLog, y cada HabitLog apunta de vuelta al Habit). Se soluciona anotando la lista en Habit con @JsonManagedReference y la referencia en HabitLog con @JsonBackReference.

Los datos desaparecen al reiniciar el servidor

Verificar que `application.properties` tenga configurada la ruta de archivo de H2 (`spring.datasource.url=jdbc:h2:file:./data/habitdb`). Sin esto, H2 usa una base de datos en memoria que se borra al apagar el servidor.

No aparece el botón de instalar la PWA

Revisar que existan los íconos referenciados en `manifest.json` (`icon-192.png` y `icon-512.png`) dentro de `static/icons/`. Sin estos archivos, el navegador puede no ofrecer la instalación.